

智能巡检车 PRO 版实验手册

ROS2 环境获得雷达测距数据

目录

1 ROS2 中部署雷达功能包	2
2 接入雷达硬件进行测试	2
3 代码实现雷达 client 节点	6
4 雷达测距相关消息结构	9

1

ROS2 中部署雷达功能包

在前面的章节中，我们已经知道如何构建 ros2 环境。接下来我们在 ros2 环境中构建雷达包。

1、执行如下指令，创建工作空间：

```
mkdir -p ~/ros2_ws/src  
cd ~/ros2_ws/src
```

2、从本系列课程提供的资源目录下的“源码”路径下获取雷达的 ros2 源码包

sllidar_ros2.tar 上传到~/ros2_ws/src 目录下，并解压缩：

```
tar xvf sllidar_ros2.tar  
source /opt/ros/foxy/setup.bash
```

3、构建 sllidar_ros2 包：

```
colcon build --symlink-install
```

4、功能包环境变量设置：

```
source ./install/setup.bash
```

2

接入雷达硬件进行测试

接入雷达硬件，并运行 ros2 应用进行测试

1、将雷达的 USB 线接到主机系统上。

2、使用如下命令为雷达的串口赋权限：

```
sudo chmod 777 /dev/ttyUSB0
```

3、使用如下指令将雷达串口绑定到固定名称：

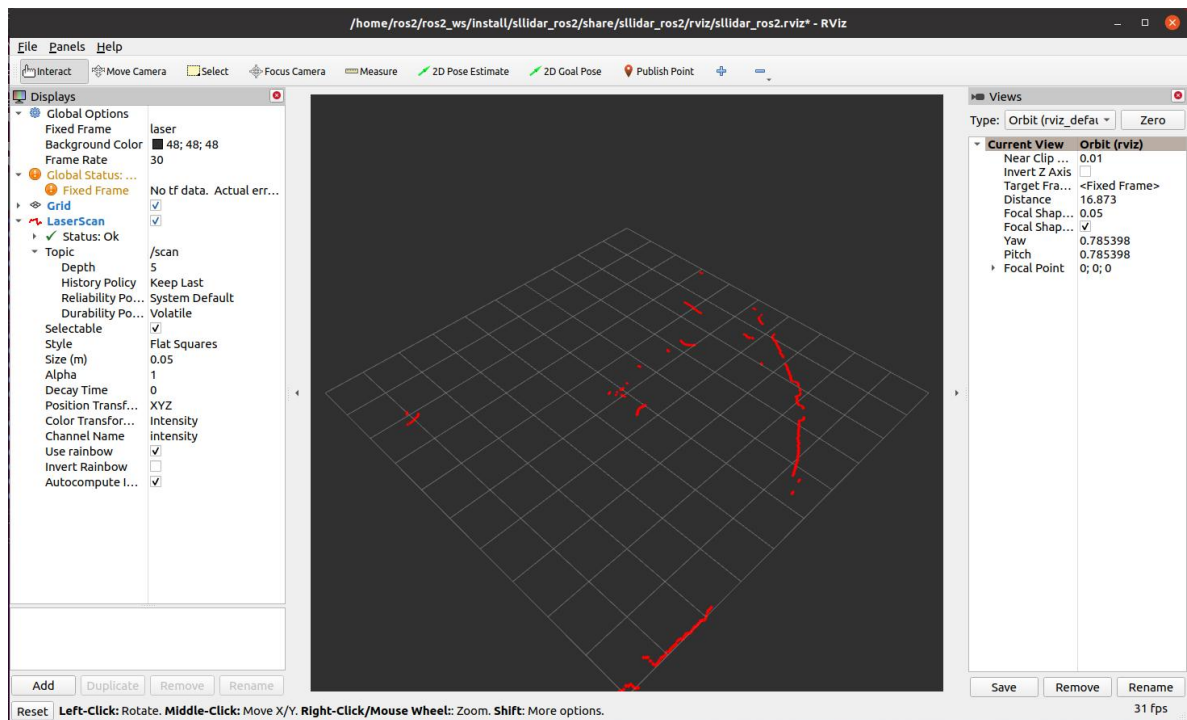
```
cd src/rplidar_ros/
source scripts/create_udev_rules.sh
sudo udevadm control --reload-rules
sudo udevadm trigger
```

此时使用如下指令，可以看到输出：

```
ros2@ubuntu:~/ros2_ws/src/sllidar_ros2/launch$ ll /dev|grep rplidar
lrwxrwxrwx    1 root root          7 Sep 17 04:22 rplidar -> ttyUSB0
```

4、运行雷达节点采集数据，并在 rviz 中显示数据

```
ros2 launch sllidar_ros2 view_sllidar_a1_launch.py
```

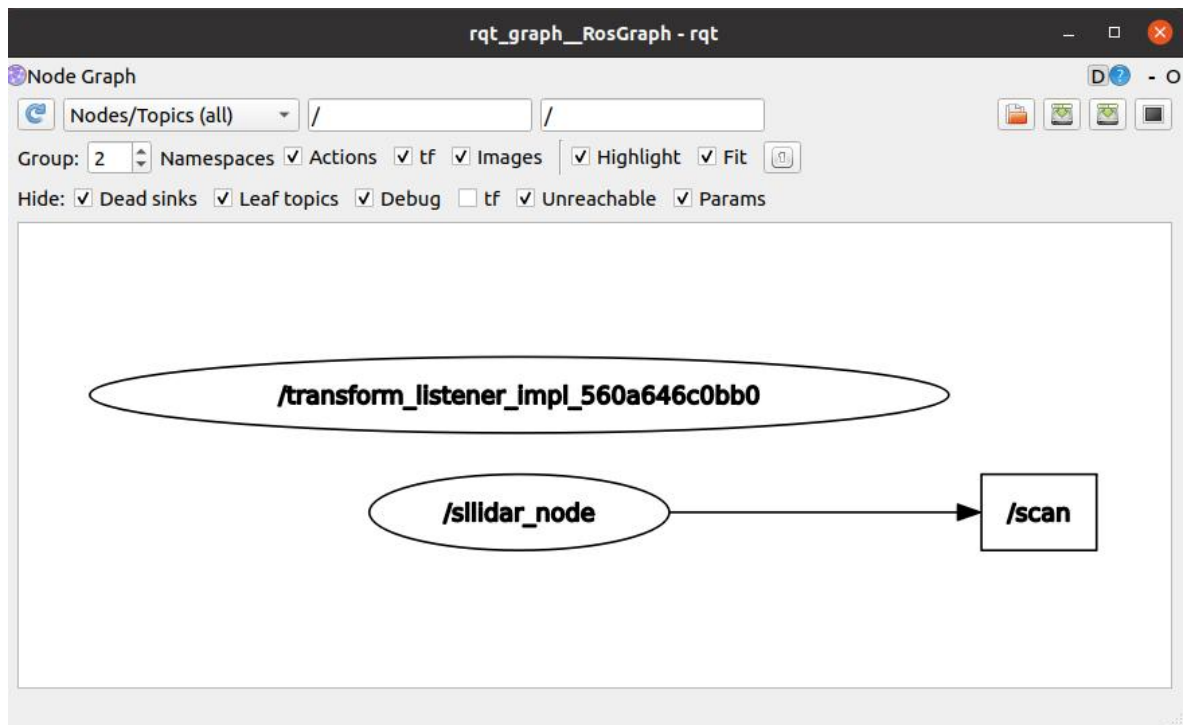


Rviz 中的红色点就是雷达扫描出来的测距数据的展示。

rqt_graph 可以用了查看当前域内运行了哪些节点以及节点之间的话题通讯。

执行如下指令运行 rqt_graph 工具：

```
ros2 run rqt_graph rqt_graph
```



可以看到雷达节点，将扫描到的数据以/scan 话题的方式发布出来。

5、启动雷达 client 节点，获取雷达测距数据

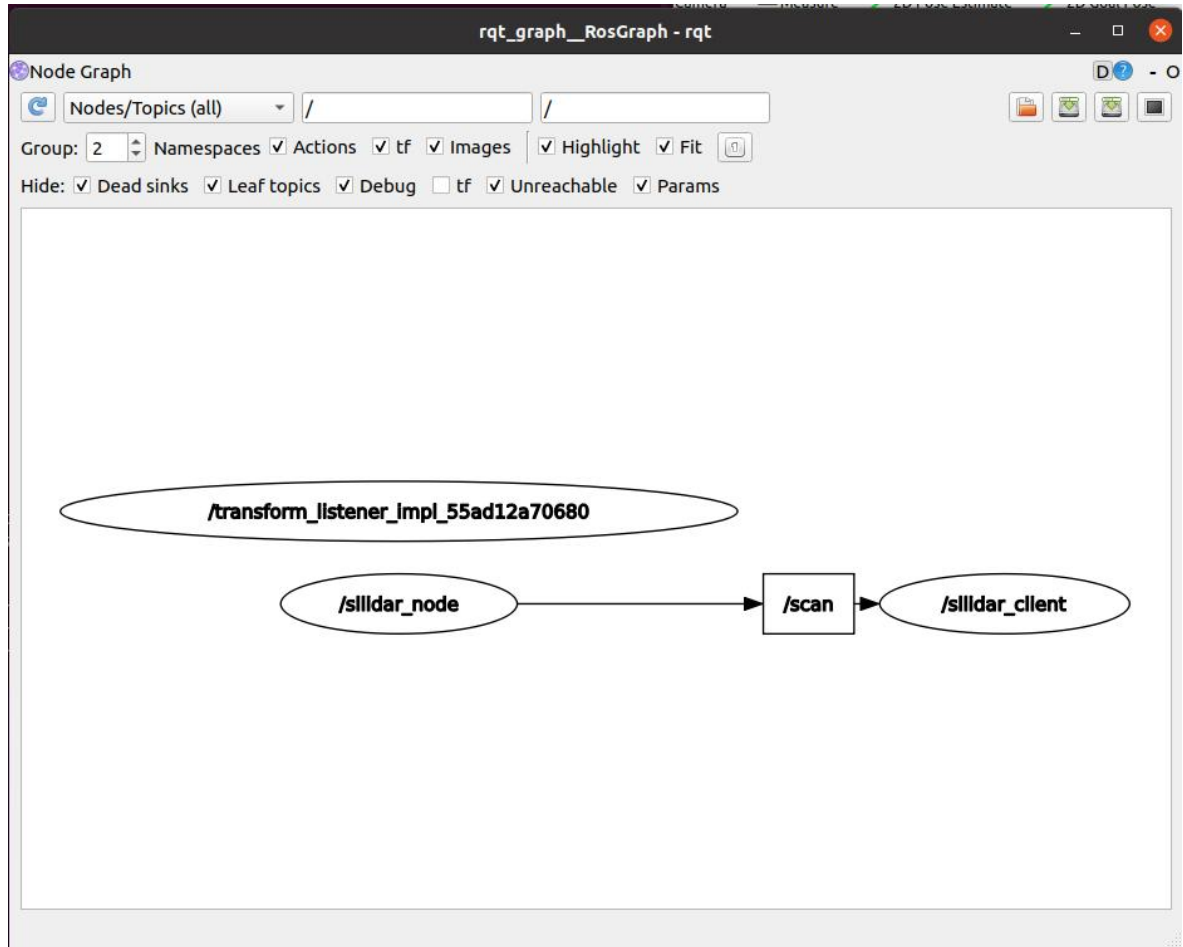
再起一个 Terminal，输入如下指令：

```
ros2 run sllidar_ros2 sllidar_client
```

将启动一个客户端节点，从/scan 话题获得具体的雷达测距数据：

```
ros2@ubuntu: ~/Desktop
[SLIDAR INFO]: angle-distance : [171.992584, 0.125000]
[SLIDAR INFO]: angle-distance : [172.326233, 0.125000]
[SLIDAR INFO]: angle-distance : [172.659866, 0.124000]
[SLIDAR INFO]: angle-distance : [172.993515, 0.124000]
[SLIDAR INFO]: angle-distance : [173.327179, 0.124000]
[SLIDAR INFO]: angle-distance : [173.660828, 0.124000]
[SLIDAR INFO]: angle-distance : [173.994461, 0.124000]
[SLIDAR INFO]: angle-distance : [174.328110, 0.124000]
[SLIDAR INFO]: angle-distance : [174.661743, 0.124000]
[SLIDAR INFO]: angle-distance : [174.995392, 0.124000]
[SLIDAR INFO]: angle-distance : [175.329025, 0.124000]
[SLIDAR INFO]: angle-distance : [175.662674, 0.124000]
[SLIDAR INFO]: angle-distance : [175.996307, 0.123000]
[SLIDAR INFO]: angle-distance : [176.329956, 0.123000]
[SLIDAR INFO]: angle-distance : [176.663589, 0.123000]
[SLIDAR INFO]: angle-distance : [176.997238, 0.122000]
[SLIDAR INFO]: angle-distance : [177.330872, 0.122000]
[SLIDAR INFO]: angle-distance : [177.664520, 0.122000]
[SLIDAR INFO]: angle-distance : [177.998154, 0.122000]
[SLIDAR INFO]: angle-distance : [178.331802, 0.122000]
[SLIDAR INFO]: angle-distance : [178.665436, 0.122000]
[SLIDAR INFO]: angle-distance : [178.999084, 0.121000]
[SLIDAR INFO]: angle-distance : [179.332718, 0.121000]
```

从 rqt_graph 工具上可以看到 slidar_client 节点从/slidar_node 节点发布的/scan 话题
获取数据。



3

代码实现雷达 client 节点

可以自己实现雷达的 client 节点，来使用 /scan 数据进行雷达测距：

1、进入源码目录

```
cd ~/ros2_ws/src
```

2、创建功能包

```
ros2 pkg create pkg_lidar_cpp --build-type ament_cmake --dependencies rclcpp  
sensor_msgs --node-name lidar_client
```

3、编辑代码

```
vi ~/ros2_ws/src/pkg_lidar_cpp/src/lidar_client.cpp
```

```
#include "rclcpp/rclcpp.hpp"
#include "sensor_msgs/msg/laser_scan.hpp"
#include <math.h>

#define RAD2DEG(x) ((x)*180./M_PI)

static void scanCb(sensor_msgs::msg::LaserScan::SharedPtr scan) {
    int count = scan->scan_time / scan->time_increment;
    printf("[SLLIDAR INFO]: I heard a laser scan %s[%d]:\n",
scan->header.frame_id.c_str(), count);
    printf("[SLLIDAR INFO]: angle_range : [%f, %f]\n",
RAD2DEG(scan->angle_min),
RAD2DEG(scan->angle_max));

    for (int i = 0; i < count; i++) {
        float degree = RAD2DEG(scan->angle_min + scan->angle_increment *
i);
        printf("[SLLIDAR INFO]: angle-distance : [%f, %f]\n", degree,
scan->ranges[i]);
    }
}

int main(int argc, char **argv) {
    rclcpp::init(argc, argv);

    auto node = rclcpp::Node::make_shared("lidar_client");

    auto lidar_info_sub =
node->create_subscription<sensor_msgs::msg::LaserScan>(
    "scan", rclcpp::SensorDataQoS(), scanCb);

    rclcpp::spin(node);

    rclcpp::shutdown();

    return 0;
}
```

4、编译功能包

cd ~/ros2_ws/
colcon build

5、生效变量

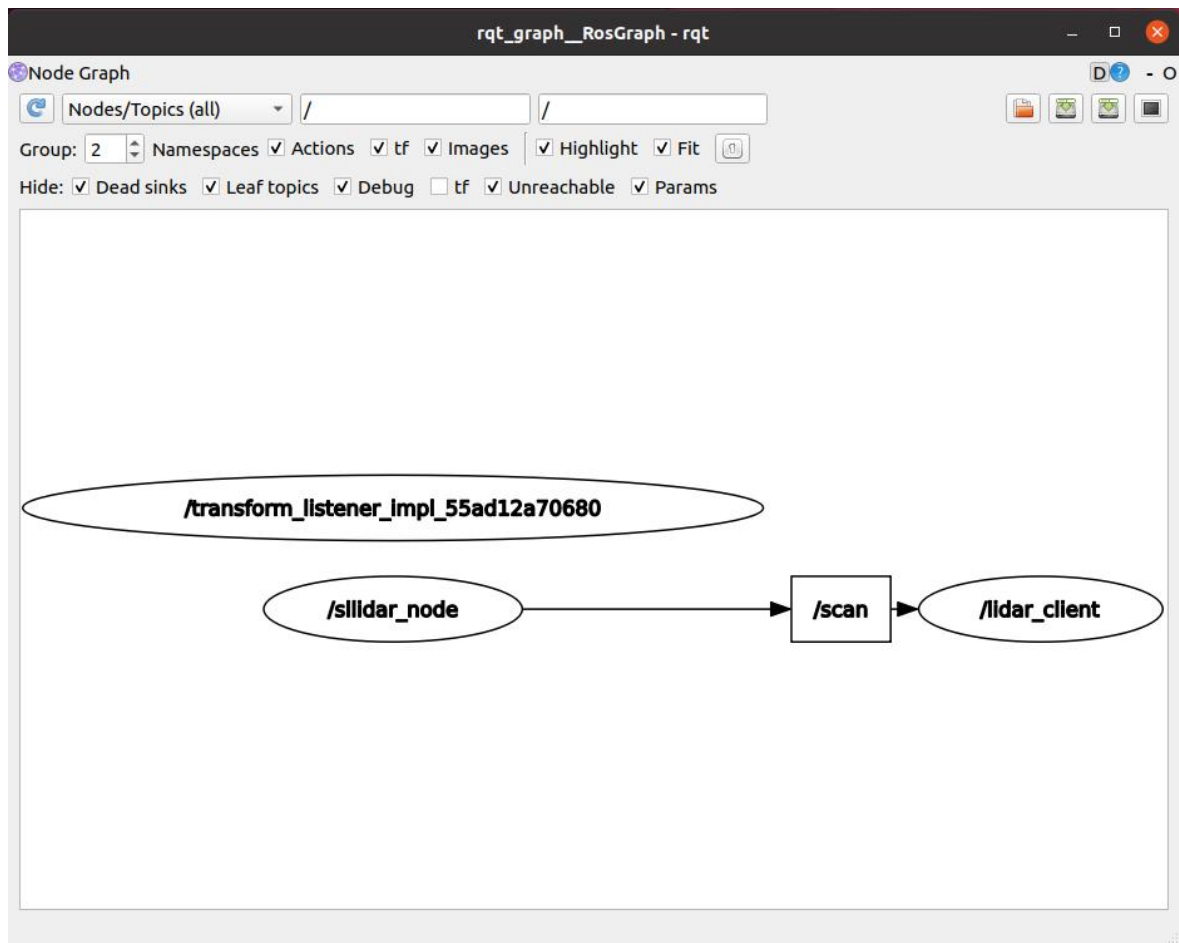

```
source ./install/setup.bash
```

6、运行功能包

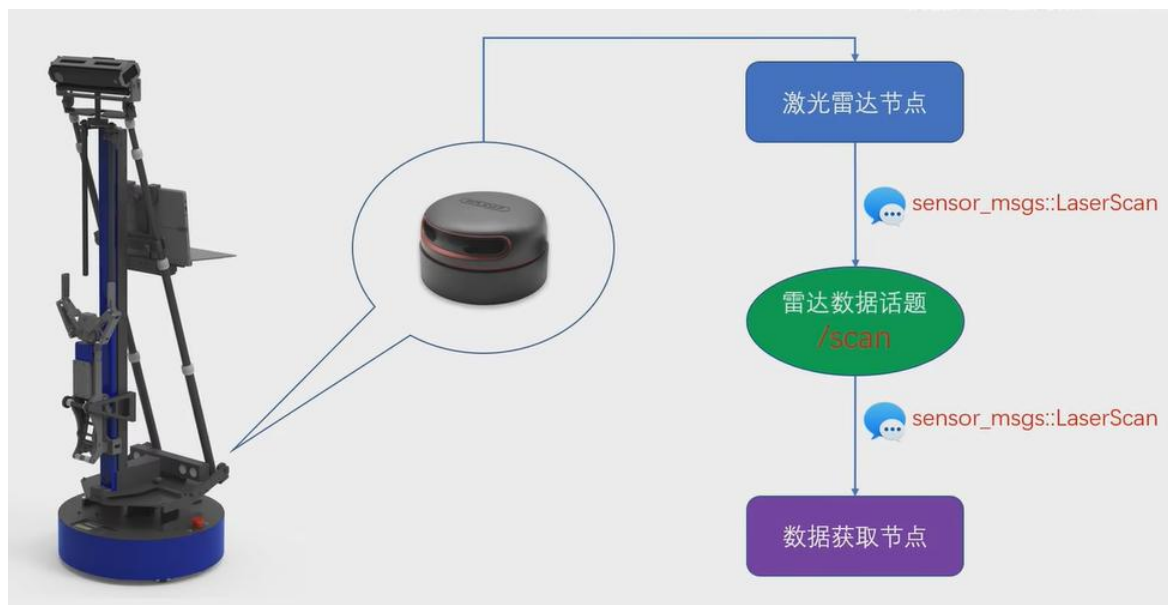
```
ros2 run pkg_lidar_cpp lidar_client
```

7、使用 rqt_graph 工具查看话题传递关系

用 rqt_graph 工具来看，可以看到 lidar_client 节点从 slidar_node 节点发布的 /scan 数据中获取雷达扫描数据。



雷达避障应用节点的程序逻辑，就是从雷达节点发布的 /scan 话题中获取雷达数据，并进行角度及距离判断，向底盘下发相关的运动指令，从而避开障碍。



4

雷达测距相关消息结构

使用 `ros2 topic info` 指令可以查看雷达测距相关的消息构成：

```
ros2@ubuntu:~$ ros2 topic info /scan
```

```
Type: sensor_msgs/msg/LaserScan
```

```
Publisher count: 1
```

```
Subscription count: 1
```

```
ros2@ubuntu:~/ros2_ws/src$ ros2 topic echo /scan --no-arr
```

```
header:
```

```
stamp:
```

```
sec: 1726664126
```

```
nanosec: 307939231
```

```
frame_id: laser
```

```
angle_min: -3.1415927410125732
```

```
angle_max: 3.1415927410125732
```

```
angle_increment: 0.005823156330734491
```

```
time_increment: 0.00011336508759995922
```

```
scan_time: 0.12232093513011932
```

```
range_min: 0.05000000074505806
```

```
range_max: 12.0
ranges: '<sequence type: float, length: 1080>'
intensities: '<sequence type: float, length: 1080>'
```

在 ROS 官方网站 index.ros.org 上可以查看 LaserScan 的详细构成:

```
std_msgs/Header header
float32 angle_min
float32 angle_max
float32 angle_increment
float32 time_increment
float32 scan_time
float32 range_min
float32 range_max
float32[] ranges
float32[] intensities
```

其中 header 的构成:

```
uint32 seq
time stamp
string frame_id
```

相关字段说明:

```
# Single scan from a planar laser range-finder
#
# If you have another ranging device with different behavior (e.g. a sonar
# array), please find or create a different message, since applications
# will make fairly laser-specific assumptions about this data

Header header          # header中的timestamp (时间戳) 表示本次扫描中接收到第一束测距激光反射信号的时间
                        #
                        # header中的frame_id指明了激光雷达的基准坐标系,
                        # 后续内容关于角度方向的描述, 都以这个坐标系为基准。
                        #
float32 angle_min      # 扫描的起始角度 (单位: 弧度)
float32 angle_max      # 扫描的终止角度 (单位: 弧度)
float32 angle_increment # 相邻两次测距的旋转夹角 (单位: 弧度)

float32 time_increment  # 相邻两次测距的时间差 (单位: 秒)
                        #
float32 scan_time       # 两次扫描的起始时间差 (单位: 秒)

float32 range_min       # 有效测距范围的最小距离 (单位: 米)
float32 range_max       # 有效测距范围的最大距离 (单位: 米)

float32[] ranges        # 本次扫描的所有测距值 (单位: 米)
float32[] intensities   # 所有测距的返回信号强度 (单位由厂商自己定义)
```